Environment Setup

SQL Server, SSMS and the course databases, on Windows, Mac and Linux

This guide follows the Environment Setup lesson step by step. It adds what the lesson could not cover: the newer screens, the other ways to run SQL Server, and the common errors with their fixes.

1 Choose your setup

Path	For	Same screens as the videos	Cost	What it needs
SQL Server Express + SSMS (section 3)	Windows	yes	free	administrator rights on the PC, 6 GB of disk
Docker (section 5)	Mac, Linux, Windows	the same SQL in Visual Studio Code	free	Docker Desktop, 4 GB of RAM free for SQL Server
Parallels with Windows (section 6)	Mac	yes, SSMS inside Windows	about 100 dollars a year, with a 14-day trial; Windows needs its own licence	16 GB of RAM, 40 GB of disk, more steps

You do not need all of it. Windows users read sections 3 and 4. Mac users read section 5, or section 6 for Windows inside the Mac. Section 10 is for everyone, when something breaks.

My advice: on Windows, take the first path, it is the one in the videos. On a Mac, take Docker: free, light, and the SQL is the same; only the tool around it changes. Take Parallels if you want SSMS exactly as in the videos and your Mac has the memory for it.

2 What you get in the materials

Download the course materials once. You get this folder:

```
sql-ultimate-course/
├── datasets/
│   ├── sql-server/     init-sqlserver-mydatabase.sql, init-sqlserver-salesdb.sql,
│   │                   MyDatabase.bak, SalesDB.bak
│   ├── mysql/          the same two databases for MySQL
│   ├── postgres/       the same two databases for PostgreSQL
│   └── *.csv           the five SalesDB tables as CSV files
├── docker/             the one-command setup for Docker (section 5)
├── docs/               the slides of every chapter as PDF
└── scripts/            every query from the course, one file per chapter
```

The course uses two databases:

Database	Used in
`MyDatabase`	SELECT, DDL, DML, Filtering Data, Joins, String Functions, Aggregate Functions
`SalesDB`	everything else, from Set Operators to AI and SQL

Every script in `scripts/` starts with a `USE` line, so it tells you which database it needs.

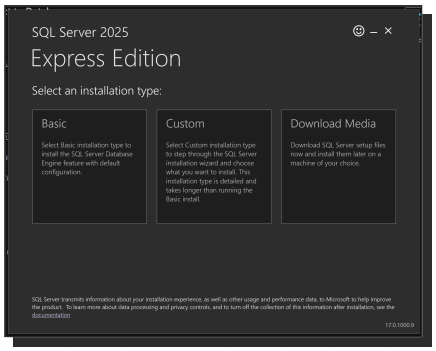
3 Windows: install SQL Server and SSMS

The course runs on SQL Server Express, the free edition. The videos show SQL Server 2022. The download page now offers SQL Server 2025 Express, and it works the same for the course.

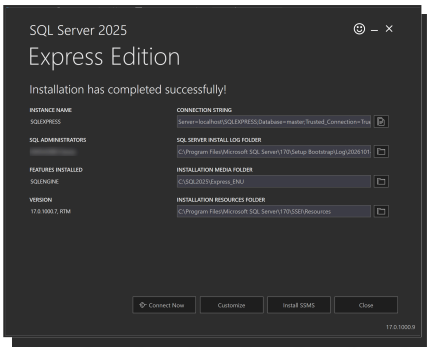
1. Open the SQL Server downloads page at microsoft.com, scroll to the **Express** card, and click **Download now**.
2. Run the downloaded file and choose **Basic**. Accept the licence and click **Install**. The installer downloads SQL Server, so this takes a few minutes.
3. The last screen shows your connection details. The server name is `localhost\SQLEXPRESS`. Keep it, you need it in a minute. The installer also made your Windows account the administrator of this SQL Server.
4. Download **SQL Server Management Studio** (SSMS) from the same page, under "SQL tools", or search for "download SSMS". Run it and keep the default options. SSMS 21 and newer install through the Visual Studio Installer, and it may offer extra components. You need none of them.



Step 1: the Express card.



Step 2: choose Basic.

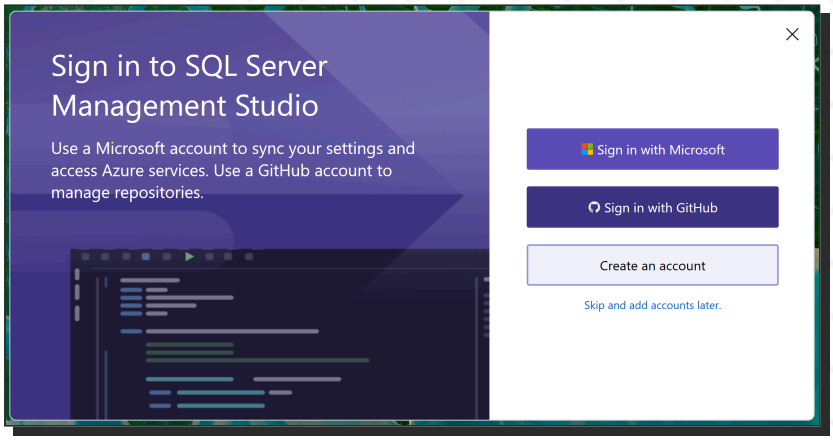


Step 3: the server name.

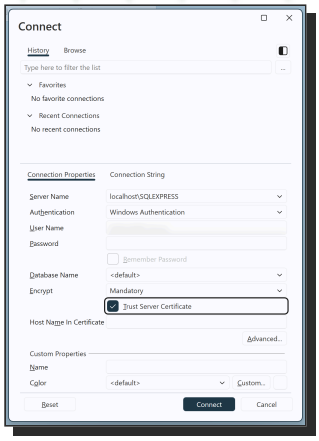
Connect for the first time

SSMS 21 and newer look different from the video.

- At the first start, SSMS asks you to sign in with a Microsoft or GitHub account. You do not need an account. Click **Skip and add accounts later** under the buttons.
- A **Connect** window opens. Type the server name `localhost\SQLEXPRESS`. If the field is empty, that is normal, just type it.
- Authentication: **Windows Authentication**.
- Tick **Trust Server Certificate**. Newer SSMS versions encrypt every connection, and without this box you get a certificate error.
- Click **Connect**. On the left, in the Object Explorer, you see your server.



The first start: the skip link under the buttons.



The Connect window, filled in.

DATA WITH BARAA

4 Create the databases

The lesson shows three ways. The first one is the easiest and works on every version.

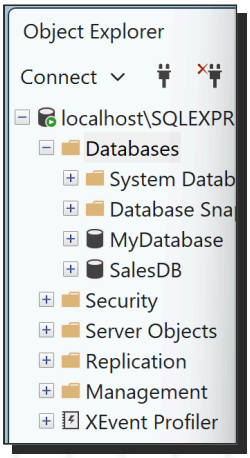
Way 1: run the scripts

1. In SSMS, click **File > Open > File** and open `datasets/sql-server/init-sqlserver-mydatabase.sql`.
2. Click **Execute** (or press F5). Refresh the Databases folder, and `MyDatabase` is there.
3. Do the same with `init-sqlserver-salesdb.sql`. Before you run it, make sure the database dropdown in the toolbar says `master`.

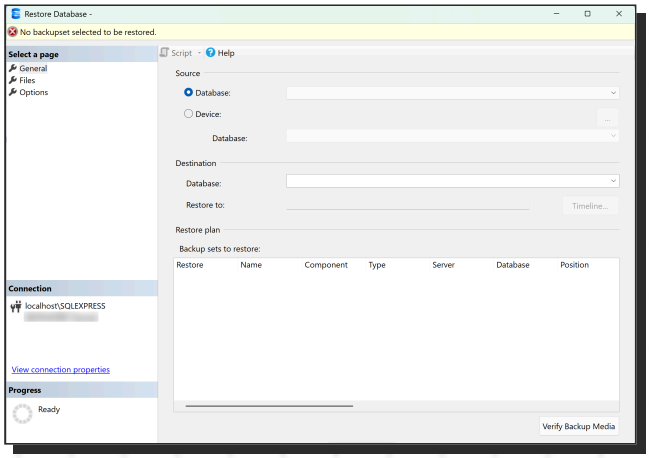
Use only the scripts in `sql-server/`. The scripts in `mysql/` and `postgres/` are for other database engines, and SSMS cannot run them.

Way 2: restore the backups

1. Copy `MyDatabase.bak` and `SalesDB.bak` into the Backup folder of your SQL Server. For SQL Server 2022 Express it is `C:\Program Files\Microsoft SQL Server\MSSQL16.SQLEXPRESS\MSSQL\Backup`. For SQL Server 2025 the folder starts with `MSSQL17`. SQL Server cannot read your Downloads folder, so this copy step matters.
2. In the Object Explorer, right-click **Databases > Restore Database**.
3. Choose **Device**, click the three dots, click **Add**, pick the backup file, and click **OK** twice.



Way 1 done: both databases in the Object Explorer.



Way 2: Device, the three dots, Add.

> One rule to know: a backup restores on the same SQL Server version or a newer one, never on an older one. The course backups come from SQL Server 2022. On SQL Server 2019 or older, use way 1.

Way 3: import the CSV files

Right-click a database, then **Tasks > Import Flat File**, and pick a CSV file. The lesson shows this as a demo. For the course, use way 1 or way 2: a CSV file cannot tell an empty text from NULL, and the NULL lessons need that difference.

5 Docker: Mac, Linux and Windows

SQL Server does not install on macOS, and SSMS is Windows only. With Docker, SQL Server runs in a small Linux container on any computer, and you write your queries in Visual Studio Code. The SQL is the same. The course materials bring a one-command setup for it.

Install

1. Install **Docker Desktop** from docker.com. On a Mac, take the Apple chip version. Start it once and leave it running.
2. On a Mac, open the Docker Desktop settings, **General**. Set the Virtual Machine Manager to **Apple Virtualization framework** and turn on **Use Rosetta for x86_64/amd64 emulation on Apple Silicon**. SQL Server ships for Intel chips only, and this setting lets your Mac run it.
3. Install **Visual Studio Code** and, inside it, the extension **SQL Server (mssql)** by Microsoft. This is Microsoft's tool for Mac and Linux. Azure Data Studio was the old one, and Microsoft retired it in February 2026.

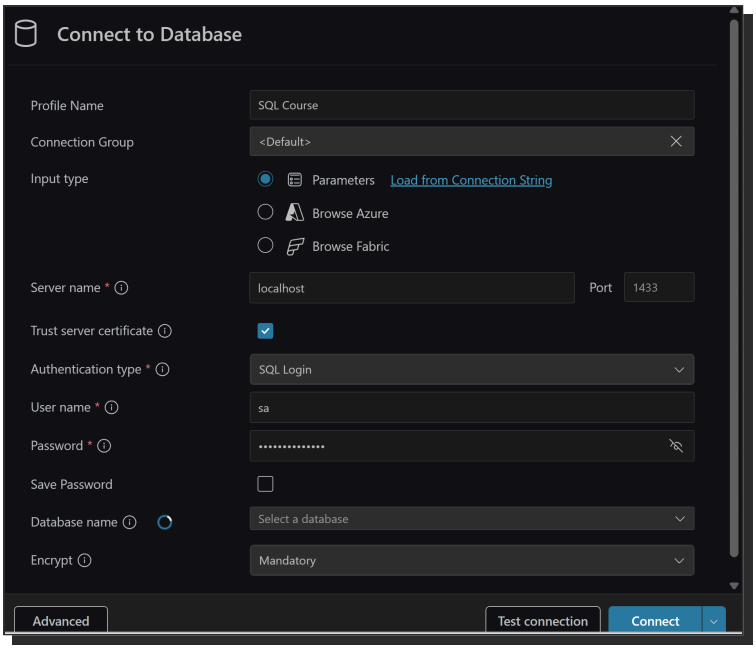
Start the course databases: one click

In the course materials, open the folder `docker` and double-click `start.command` on a Mac or `start.bat` on Windows. On Linux, run `docker compose up -d --build` in that folder. The first start downloads SQL Server, about 1.5 GB, and creates `MyDatabase` and `SalesDB` from the course scripts. After that, a container called `sql-course` sits in Docker Desktop, and you start it from there whenever you want to practice.

Then connect in Visual Studio Code. Open the SQL Server extension on the left and add a connection:

- Server name `localhost`, port `1433` (in SSMS this is one field: `localhost,1433`)
- Tick **Trust server certificate**
- Authentication type **SQL Login**, user `sa`, password `SqlCourse!2026`

That is all. Open a script from `scripts/`, run it against the connection, and you are in the course.



The connection form in Visual Studio Code, filled in.

The same by hand

If you prefer one command of your own, this starts an empty SQL Server 2022. Choose your own password: at least 8 characters with upper case, lower case, a number and a symbol.

```
docker run -e "ACCEPT_EULA=Y" -e "MSSQL_SA_PASSWORD=YourStrong!Passw0rd" \
  -p 1433:1433 --name sql --platform linux/amd64 \
  -d mcr.microsoft.com/mssql/server:2022-latest
```

Then connect as above and run `init-sqlserver-mydatabase.sql` and `init-sqlserver-salesdb.sql` from Visual Studio Code.

The projects in Docker

The two projects load CSV files with `BULK INSERT`, and the scripts point to Windows paths. The one-click setup solves this for you. Unzip the three packages next to each other, like this. Then the project folders are visible inside the container under `/var/opt/mssql/sql/`, the same layout as `C:\sql\` in the videos.

```
sql-course/
├── sql-ultimate-course/
├── sql-data-warehouse-project/
└── sql-data-analytics-project/
```

In the project scripts, replace `C:\sql\` with `/var/opt/mssql/sql/` and the backslashes with slashes. `C:\sql\dwh_project\datasets\source_crm\cust_info.csv` becomes `/var/opt/mssql/sql/dwh_project/datasets/source_crm/cust_info.csv`. With the analytics project folder in place, the one-click setup even creates the `DataWarehouseAnalytics` database for you.

6 Windows inside your Mac: Parallels

This path gives you SSMS exactly as in the videos, because you run Windows on your Mac. **Parallels Desktop** (parallels.com) downloads and installs Windows 11 for you in a few clicks. It costs about 100 dollars a year, with a 14-day free trial, and Windows needs its own licence. Give the virtual machine at least 6 GB of memory, so a Mac with 16 GB is the comfortable minimum.

Inside Windows, follow section 3 like any Windows user. SQL Server Express installs and runs on the ARM version of Windows 11, even though Microsoft does not list it as supported. Tested with SQL Server 2025 Express and SSMS 22 in Parallels on an Apple chip.

7 MySQL and PostgreSQL

The course is recorded on SQL Server, and that is the engine I recommend: every query in the videos runs as shown. If you still want MySQL or PostgreSQL, here is what to install, how to load the two databases, and the differences you will meet.

Install and load

Engine	Download	Tool to write queries	Load the course databases
MySQL	MySQL Community Server, from dev.mysql.com. The MySQL Installer on Windows, the DMG on a Mac	MySQL Workbench, from the same page	In Workbench: **File > Open SQL Script**, open `datasets/mysql/init-mysql-mydatabase.sql`, run it with the lightning icon. Then the same with `init-mysql-salesdb.sql`
PostgreSQL	The EDB installer from postgresql.org, for Windows and Mac. On a Mac, Postgres.app is the lighter choice	pgAdmin, which comes with the EDB installer. Or DBeaver, or Postico on a Mac	Open `datasets/postgres/init-postgres-mydatabase.sql` in the query tool while connected to the `postgres` database, and run the lines up to `CREATE DATABASE`. Then connect to the new database and run the rest. The same with `init-postgres-salesdb.sql`

Both engines keep the table names in lower case: `customers`, `orders`, `orders_archive` in MySQL, and `sales.customers` in PostgreSQL.

What is different

Most of the course runs unchanged: SELECT, filtering, joins, set operators, CASE, aggregations, window functions, subqueries, CTEs, views. These are the parts that differ, and how to write them on the other engines.

In the course (SQL Server)	MySQL	PostgreSQL
`SELECT TOP 3 * FROM t`	`SELECT * FROM t LIMIT 3`	`SELECT * FROM t LIMIT 3`
`GETDATE()`	`NOW()`	`NOW()`
`DATETRUNC(month, d)`	`DATE_FORMAT(d, '%Y-%m-01')`	`DATE_TRUNC('month', d)`
`DATENAME(month, d)`	`MONTHNAME(d)`	`TO_CHAR(d, 'Month')`
`DATEPART(year, d)`, `YEAR(d)`	`YEAR(d)`	`EXTRACT(YEAR FROM d)`
`EOMONTH(d)`	`LAST_DAY(d)`	`(DATE_TRUNC('month', d) + INTERVAL '1 month - 1 day')::date`
`DATEADD(day, 7, d)`	`DATE_ADD(d, INTERVAL 7 DAY)`	`d + INTERVAL '7 days'`
`DATEDIFF(day, a, b)`	`DATEDIFF(b, a)`, days only	`b - a` for dates
`FORMAT(d, 'yyyy-MM-dd')`	`DATE_FORMAT(d, '%Y-%m-%d')`	`TO_CHAR(d, 'YYYY-MM-DD')`
`CONVERT(DATE, x)`	`CAST(x AS DATE)`	`CAST(x AS DATE)` or `x::date`
`ISNULL(a, b)`	`IFNULL(a, b)` or `COALESCE(a, b)`	`COALESCE(a, b)`
`IIF(cond, a, b)`	`IF(cond, a, b)`	`CASE WHEN cond THEN a ELSE b END`
`LEN(s)`	`CHAR_LENGTH(s)`	`LENGTH(s)`
`CHARINDEX(x, s)`	`LOCATE(x, s)`	`POSITION(x IN s)`
`'a' + 'b'` for text	`CONCAT('a', 'b')`	`'a' || 'b'`
`[Column Name]`	`` `Column Name` ``	`"Column Name"`
`Sales.Customers`	`customers`, MySQL has no schemas inside a database	`sales.customers`
`#temp` tables	`CREATE TEMPORARY TABLE`	`CREATE TEMP TABLE`
`SELECT ... INTO new_table`	`CREATE TABLE new_table AS SELECT ...`	`CREATE TABLE new_table AS SELECT ...`
`BULK INSERT`	`LOAD DATA INFILE`	`COPY ... FROM`

The chapters on stored procedures, triggers, indexes, partitions, execution plans and performance tips are SQL Server chapters. The ideas carry over, the syntax does not. When a query from those chapters fails on another engine, that is the reason, not a mistake in your setup.

8 The project datasets

Data Warehouse project

The six CSV files sit in `datasets/source_crm` and `datasets/source_erp`. The bronze loader reads them from `C:\sql\dwh_project\datasets\`, the folder used in the videos. Put the folder there, or change the six paths in `proc_load_bronze.sql`. The folder must be readable by the SQL Server service, so a folder directly on `C:\` works, and Downloads often does not.

Data Analytics project

Restore `DataWarehouseAnalytics.bak` the same way as in section 4, or run `00_init_database.sql`, which loads the gold tables from `datasets/csv-files/`. If you built the data warehouse project, you already have the gold layer.

9 AdventureWorks

The index chapter uses Microsoft's sample database `AdventureWorksDW2022`. It is not part of the course materials because of its size. Get it from Microsoft: search for "AdventureWorks sample databases", open the Microsoft Learn page, and download `AdventureWorksDW2022.bak`, the data warehouse with `FactInternetSales`. `AdventureWorks2022.bak` is the other, operational database, and it does not have that table.

Restore it like the course backups. The 2022 files need SQL Server 2022 or newer. On SQL Server 2019, take the 2019 files from the same page: the data is the same.

10 When something breaks

Find your message in the first column. The fix is in the last one.

What you see	Why	What to do
SSMS asks you to sign in at the first start	optional account	Click **Skip and add accounts later**
The server name field is empty	normal	Type `localhost\SQLEXPRESS`
Error 26, "Error Locating Server/Instance Specified"	SQL Server is not running, or the name is wrong	Open **Services**, start "SQL Server (SQLEXPRESS)". If you installed the Developer edition, the name is `localhost` without the instance
"The certificate chain was issued by an authority that is not trusted"	SSMS encrypts the connection	Tick **Trust Server Certificate**
Msg 262, "CREATE DATABASE permission denied in database 'master'"	your Windows login is not an administrator of this SQL Server. Someone else installed it, often IT	Log in with the Windows account that installed SQL Server, or ask an administrator to add your login to the **sysadmin** role (Security > Logins > Properties > Server Roles)

What you see	Why	What to do
Msg 911, "Database 'MyDatabase' does not exist", or many syntax errors with backticks	you ran the MySQL or PostgreSQL script in SSMS	Use the scripts in `datasets/sql-server/`
The restore window does not show the .bak file, or "Access is denied"	SQL Server cannot read Downloads	Copy the file into the Backup folder from section 4
Msg 3169, the backup "is incompatible with this server"	your SQL Server is older than 2022	Run the scripts (way 1), or install SQL Server 2022 or 2025 Express
"'DATETRUNC' is not a recognized built-in function name"	SQL Server older than 2022	Install SQL Server 2022 or newer. The course uses functions from 2022
Msg 4860, "Cannot bulk load. The file does not exist or you don't have file access rights"	the path in the script is not your folder, or SQL Server cannot read it	Put the datasets folder on `C:\` and fix the path in the script
Installer: "Wait on the Database Engine recovery handle failed"	SQL Server could not start during setup	Uninstall SQL Server, restart Windows, install again. If it fails again, the real reason is in the ERRORLOG file under `C:\Program Files\Microsoft SQL Server\MSSQL17.SQLEXPRESS\MSSQL\Log` (MSSQL16 for 2022)
SQL Server 2025 setup stops at the rules check	installer problem on some machines	Install SQL Server 2022 Express instead. The course runs on it
A .sql file opens in Notepad, or the .bak shows as text in the browser	the file is not linked to SSMS	Open SSMS first, then **File > Open > File**. For downloads, right-click the link and choose **Save link as**
Docker: "Cannot connect to the Docker daemon"	Docker Desktop is not running	Start Docker Desktop, wait for the whale icon, try again
Docker: "no matching manifest for linux/arm64"	the Intel image needs emulation	Add `--platform linux/amd64` to the command and turn on Rosetta in Docker Desktop
Docker: "port is already allocated"	another SQL Server listens on 1433	Stop it, or change the first `1433` in `docker-compose.yml` to `1434` and connect to `localhost,1434`
Docker: the container stops right after the start	the password breaks the rules	At least 8 characters with upper case, lower case, a number and a symbol
Your laptop does not let you install anything	no administrator rights	SQL Server needs them. Ask IT, or use a personal computer

If you are still stuck, write in the Q&A with the exact error message and a screenshot of the window. Then I can see what happened.